

LLM-ERP 系統操作手冊

以大語言模型驅動的智慧企業資源規劃系統
版本：v0.1.0 | 更新日期：2026-05-07

1. 系統概述

LLM-ERP 是一套開源的智慧 ERP 系統，讓使用者透過**自然語言**管理整個工廠流程，不需要記憶 T-code 或點選單。系統涵蓋 13 大模組：

模組	代碼	核心功能
 自然語言介面	—	支援中文與英文，系統自動判斷語言回應
 庫存管理	MM	料號管理、庫存查詢、入出庫異動、儲位追蹤
 採購管理	PP	供應商管理、採購單生命週期、供應商評分
 BOM 工程	ENG	產品結構管理、多階展開、缺料檢查
 生產派工	MFG	工單管理、機台排程、動態重排程（3 策略）
 品質管理	QM	檢驗單管理、不合格品(NC)追蹤、矯正措施(CAPA)
 會計財務	FI	會計科目、傳票/分錄、AR 逾期管理、月結
 報表生成	—	中英文指令產出 PDF 報表（庫存/AR/採購/生產/損益）
	 戰情室	—
	 CRM 客戶管理	SD
	 潛在客戶管理	LE
	 商機管理	OP
	 合約管理	CT
	 決策記錄與 AAR	AR
	 現金流管理	CF
	 工廠型態設定	FC
	 7 大角色	—

2. 安裝需求

硬體需求

配置	最低	建議
RAM	4 GB	16 GB（含本地 LLM）
CPU	2 cores	8 cores
本地 LLM	—	16 GB RAM + GPU（可選）

軟體需求

- Python 3.11+
- Node.js 18+
- (選用) Docker Desktop — 用 PostgreSQL 正式環境
- (選用) Ollama — 本地 LLM 推理

3. 安裝步驟

3.1 下載專案

```
git clone https://github.com/fanchanyu/llm-erp.git
cd llm-erp
```

3.2 後端設定

```
cd backend
cp .env.example .env
# 編輯 .env，填入你用的 LLM Provider 的 API Key
```

`.env` 檔案說明：

變數	說明	範例
LLM_PROVIDER	LLM 提供商	deepseek / ollama / anthropic / openai
LLM_MODEL	模型名稱	deepseek-chat / gemma4:e4b / claude-sonnet-4
MAX_TOOL_ROUNDS	工具調用輪數	5 (雲端) / 8-10 (本地)
DEEPSEEK_API_KEY	DeepSeek API Key	sk-xxxx

3.3 申請 API Key (選擇一個 Provider)

LLM-ERP 需要一個 LLM (大型語言模型) 來驅動自然語言功能。以下是各家 Provider 的申請方式：

◆ **DeepSeek (推薦, 最便宜)**

1. 前往 https://platform.deepseek.com/sign_up 註冊帳號
2. Email 驗證 → 登入
3. 點選左側「API Keys」→「創建 API Key」
4. 複製 key (格式: sk-xxxxxxxxxxxxxxxxxx)
5. 填入 `.env`: `DEEPSEEK_API_KEY=sk-xxxx`

- 費用：約 ¥1 元人民幣可處理 200~500 次查詢
- 模型：deepseek-chat
- 適合：剛開始試用、開發階段

◆ **Anthropic Claude (最聰明)**

1. 前往 <https://console.anthropic.com/> 註冊
2. 登入後點選「API Keys」→「Create Key」
3. 複製 key (格式: sk-ant-xxxxxxxxxxxxxx)
4. 填入 `.env`：
`LLM_PROVIDER=anthropic`

```
LLM_MODEL=claude-sonnet-4
ANTHROPIC_API_KEY=sk-ant-xxxx
```

- **費用** : \$3/M 輸入 + \$15/M 輸出 tokens
- **模型** : claude-sonnet-4 / claude-haiku-3-5 (便宜快速)
- **適合** : 正式生產環境、需要高準確率

◆ OpenAI GPT (最普及)

1. 前往 <https://platform.openai.com/signup> 註冊
 2. 登入 → 左上角「API Keys」→ 「Create new secret key」
 3. 複製 key (格式: sk-proj-xxxxxxxxxxx)
 4. 填入 .env :
- ```
LLM_PROVIDER=openai
LLM_MODEL=gpt-4o
OPENAI_API_KEY=sk-proj-xxxx
```

- **費用** : \$2.50/M 輸入 + \$10/M 輸出 (gpt-4o)
- **模型** : gpt-4o / gpt-4o-mini (省錢)
- **適合** : 已有 OpenAI 帳號者、通用場景

#### ◆ OpenRouter (一站多用, 可選多種模型)

1. 前往 <https://openrouter.ai/keys> 註冊
  2. 登入 → 「Create Key」
  3. 複製 key (格式: sk-or-v1-xxxxxxxxxx)
  4. 填入 .env :
- ```
LLM_PROVIDER=openrouter
LLM_MODEL=deepseek/deepseek-chat
OPENROUTER_API_KEY=sk-or-v1-xxxx
```

- **費用** : 依模型不同 (可用最便宜的)
- **模型清單** : <https://openrouter.ai/models>
- **適合** : 想比較多家模型、切換方便

◆ Ollama (本地模型, 完全免費, 不需 API Key)

如果你的電腦有 16GB+ RAM, 可以在本地跑模型, 完全不需要 API Key :

```
# 安裝 Ollama
curl -fsSL https://ollama.com/install.sh | sh

# 下載 Gemma4 模型 (8B, 約 9.6GB)
ollama pull gemma4:e4b

# 確認 Ollama 正在執行
curl http://localhost:11434/api/tags

# 設定 .env :
# LLM_PROVIDER=ollama
# LLM_MODEL=gemma4:e4b
# MAX_TOOL_ROUNDS=8
```

- **費用** : 完全免費, 只需要硬體
- **硬體需求** : 16GB RAM (8B 模型) / 32GB (12B 模型)

- 適合：資料不外洩、不想付 API 費用

⚠ **API Key 安全提醒**：Key 只能放在 `.env`，絕對不能放到 CSV 資料檔、Git 提交、或貼到 GitHub 上。

3.8 啟動後端

```
cd backend
pip install -r requirements.txt
uvicorn app.main:app --reload
```

後端會在 `http://localhost:8000` 啟動。

3.9 啟動前端

```
cd frontend
npm install
npm run dev
```

前端會在 `http://localhost:5173` 啟動。

3.10 啟動本地 LLM（選用）

如果你選擇使用本地模型（Ollama），請先參考 [3.3 節 — Ollama 申請方式](#) 安裝 Ollama 並下載模型，然後確認 Ollama 正在執行：

```
curl http://localhost:11434/api/tags
```

之後回到 [3.8 節](#) 啟動後端即可。

3.11 資料管理 (Import / Export / Reset)

LLM-ERP 提供標準化的資料匯入匯出工具，所有資料使用**自然鍵**（料號、供應商名稱、產品編號等）—— 不是 UUID，所以人類可以直接編輯 CSV。

API Key 永遠不會出現在資料檔中，只存在於 `.env`。

匯入資料

```
cd backend

# 匯入內建範例資料 (12 張表, 71 筆記錄)
python -m scripts.manage_data import scripts/sample_data/

# 或用自己的 CSV
python -m scripts.manage_data import path/to/my-data.csv

# 或匯入整個目錄 (自動按依賴順序處理)
python -m scripts.manage_data import path/to/data-dir/

# 乾執行 (只驗證, 不改資料)
python -m scripts.manage_data import scripts/sample_data/ --dry-run
```

匯出資料

```
python -m scripts.manage_data export ./backup/
# 輸出：01-parts.csv ~ 12-ar.csv , 共 12 張表
```

重置資料庫

```
python -m scripts.manage_data reset --force
# 清空所有 22 張表
```

查看 Schema

```
python -m scripts.manage_data schema
# 列出所有實體的欄位定義
```

CSV Schema 速查表

檔案	實體	必要欄位	依賴
01-parts.csv	料號主檔	part_no, name, unit	—
02-suppliers.csv	供應商	name	—
03-products.csv	產品	product_no, name	—
04-work-centers.csv	工作站	name	—
05-accounts.csv	會計科目	account_no, name, type, normal_balance	—
06-inventory.csv	庫存量	part_no, location, quantity	parts
07-bom.csv	BOM 表	product_no, part_no, quantity, level	products, parts
08-purchase-orders.csv	採購單	po_no, supplier_name, item_part_no, item_quantity	suppliers, parts
09-production-orders.csv	工單	order_no, product_no, quantity, due_date	products
10-quality.csv	檢驗單	inspection_no, part_no, quantity	parts, purchase-orders
11-accounting.csv	會計傳票	entry_no, description, entry_date, period, line_account_no, line_debit, line_credit	accounts
12-ar.csv	應收帳款	customer_name, invoice_no, amount, due_date	—

匯入工具會自動依賴順序處理，跳過已存在的記錄（冪等操作）。

```
# 安裝 Ollama
curl -fsSL https://ollama.com/install.sh | sh
```

```
# 下載 Gemma4 模型 (8B , 約 9.6GB)
ollama pull gemma4:e4b

# 確認 Ollama 正在執行
curl http://localhost:11434/api/tags

# 將 .env 設為 :
# LLM_PROVIDER=ollama
# LLM_MODEL=gemma4:e4b
# MAX_TOOL_ROUNDS=8
```

4. 使用指南

4.1 庫存管理 (Inventory)

你可以問任何關於庫存的問題：

中文查詢	說明
「M6x20 螺絲還有多少庫存？」	查詢特定料號庫存
「幫我看一下全部庫存列表」	列出所有庫存
「傳動件有哪些零件？庫存夠嗎？」	按分類查詢
「查一下馬達類零件的庫存狀況」	依名稱模糊查詢
「入庫 M6x20 螺絲 500 顆」	入庫作業
「發料 M6x20 螺絲 100 顆給工單 WO-001」	出庫/領料

4.2 採購管理 (Purchase)

中文查詢	說明
「供應商有哪些？查一下大明螺絲」	查詢供應商
「幫我開一張採購單，向大明螺絲買 M6x20 200 顆」	建立採購單
「目前的採購單有哪些？」	查詢採購單列表
「PO-2026-0001 的貨到了嗎？」	查詢採購單狀態

4.3 BOM 與物料管理

中文查詢	說明
「CNC-001 用哪些料？」	查詢 BOM 結構
「CNC-001 的 BOM 展開給我看」	BOM 多階展開
「CNC-001 要做 5 台，料夠不夠？」	缺料檢查 (核心功能)

4.4 生產派工 (Dispatch)

中文查詢	說明
------	----

「目前有哪些工單？」	查詢工單列表
「幫我釋出工單 WO-20260506-001」	釋出工單
「CNC-01 故障，往後推 30 分鐘」	右移重排程 (Right-Shift)
「CNC-01 故障，換到備用機台」	替代路徑重排程 (Route Change)
「有急單 WO-002，插隊優先處理」	急單插隊 (Expedite)

4.5 品質管理 (Quality)

中文查詢	說明
「新增品檢單 M6x20，批號 LOT-001」	建立檢驗單
「查一下品檢記錄有哪些」	查詢檢驗單列表
「M6x20 有不良品嗎？」	查詢不合格品(NC)
「新增 NC，M6x20 尺寸超差，嚴重度=重大」	建立不良品記錄

4.6 會計財務 (Accounting)

中文查詢	說明
「有哪些逾期應收帳款？」	查詢逾期帳款
「查一下應收帳款報表」	查詢 AR 列表
「會計科目有哪些？」	查詢科目表
「開傳票：借庫存 1000 貸銀行存款 1000」	建立傳票/分錄

4.7 CRM 客戶管理 (Sales)

中文查詢	說明
「客戶有哪些？查一下永裕精密」	查詢客戶列表/搜索
「新增客戶：宏達電子，聯絡人張課長」	新增客戶主檔
「幫我開一張銷售訂單，永裕精密買 CNC-001 10 件」	建立銷售訂單
「目前的銷售訂單有哪些？交期如何？」	查詢 SO 列表
「SO-20260507-001 確認下去」	確認 SO → 自動開生產工單
「SO-20260507-001 出貨了」	SO 出貨 → 自動扣庫存
「記錄一下：永裕精密來電詢問交期」	新增客戶互動事件
「永裕精密的對話記錄有哪些？」	查詢客戶歷史對話

4.8 跨模組查詢

中文查詢	說明
「M6x20 螺絲採購單 PO-0001 的貨到了嗎？入庫了？」	採購+庫存
「我要生產 5 台 CNC-001，先檢查料夠不夠」	BOM+採購

4.9 潛在客戶與商機管理 (Leads & Opportunities)

管理尚未成交的客戶來源與銷售 Pipeline：

功能	說明
潛在客戶來源追蹤	展會、官網、轉介、陌生開發等來源自動分類
Lead Scoring 評分機制	根據互動行為、公司規模、預算等自動評分
Pipeline 階段管理	初步接觸→需求了解→提案→議價→談判→成交
轉換率分析	各階段轉換率、平均成交天數、丟單原因統計

LLM 問法範例：

中文查詢	說明
「有哪些潛在客戶？」	查詢所有 Lead 列表
「幫我查商機 Pipeline」	查詢各階段商機分布
「最近展會來的 Lead 有哪些？評分多少？」	按來源篩選 + 評分
「把永裕精密這個 Lead 轉成商機，金額 50 萬」	Lead → Opportunity 轉換
「哪些商機超過 30 天沒更新？」	停滯商機預警

4.10 合約管理 (Contracts)

管理與客戶簽訂的各種合約，並自動帶入銷售流程：

功能	說明
框架合約	長期合作框架，單次 SO 依框架條款執行
年度合約	每年更新價格與數量承諾
專案合約	一次性專案，含里程碑付款條款
合約價格自動帶入 SO	開 SO 時自動比對合約價格，無需人工核對
合約到期警示	到期前 30 天自動通知業務部門

LLM 問法範例：

中文查詢	說明
「有哪些有效合約？」	查詢目前生效中的所有合約
「永裕精密的合約什麼時候到期？」	查詢特定客戶合約到期日
「幫我查永裕精密的合約價格，CNC-001 單價多少？」	查詢合約價格條款
「宏達電子的一年合約快到了，幫我續約」	建立續約
「哪些合約 30 天內到期？」	到期預警查詢

4.11 決策紀錄與事後回顧 (Decision AAR)

系統自動記錄重大決策，並提供結構化的回顧框架 (After Action Review)：

功能	說明
重大決策自動記錄	急單決策、供應商變更、排程調整等自動留存
AAR 流程	預期 vs 實際 → 偏差分析 → 矯正措施 → 系統規則更新
部門 KPI 反饋閉環	決策影響回饋到部門績效指標
歷史回溯	可查詢任意時間範圍的決策記錄

LLM 問法範例：

中文查詢	說明
「業務部門最近的決策？」	查詢特定部門的決策記錄
「上個月急單的回顧報告」	查詢 AAR 報告
「記錄一下：今天決定換掉大明螺絲，改採購源興軸承」	手動記錄決策
「幫我對上過的急單做 AAR」	觸發 AAR 回顧流程
「採購部門這個月的 KPI 如何？」	KPI 反饋查詢

4.12 急單評估與現金流約束

處理插單、急單時自動評估財務影響，並監控公司現金水位：

功能	說明
急單財務影響評估	溢價收入 - 加班成本 - 延遲罰款 = 淨效益
現金水位查詢	即時可用現金 + 應收帳款 - 應付帳款
30 天現金預測	根據 SO/PO 預估未來現金流入流出
現金不足自動鎖定採購	水位低於安全線時禁止新增採購單

LLM 問法範例：

中文查詢	說明
「幫我評估這個急單：永裕精密要插單 CNC-001 × 10，客戶願付 20% 溢價」	急單財務評估
「公司現金水位如何？」	即時現金查詢
「未來 30 天現金預測？」	現金預測報表
「為什麼不能開採購單？」	現金水位不足提示

4.13 工廠型態設定 (Factory Config)

依工廠營運模式設定系統行為，支援三種型態：

型態	說明	適用場景	Pipeline 特徵
MTO (Make-to-Order)	接到訂單才生產	機械加工、CNC 銑床	接單→備料→生產→出貨，庫存維持最低
MTS (Make-to-Stock)	預測需求先備貨	電子零件、標準品	預測→生產→入庫→接單→出貨，安全庫存管控

ETO (Engineer-to-Order)	接單後設計再生產	自動化設備、專案型	RFQ→設計→議約→採購→生產→安裝，含里程碑管理
-------------------------	----------	-----------	---------------------------

設定方式 (curl 範例)：

```
# 設定為 MTO 模式
curl -X POST http://localhost:8000/api/config/factory \
  -H "Content-Type: application/json" \
  -d '{"mode": "MTO", "safety_stock_days": 0, "auto_reorder": false}'

# 設定為 MTS 模式 (電子零件廠)
curl -X POST http://localhost:8000/api/config/factory \
  -H "Content-Type: application/json" \
  -d '{"mode": "MTS", "safety_stock_days": 14, "auto_reorder": true, "reorder_point": 100}'

# 設定為 ETO 模式
curl -X POST http://localhost:8000/api/config/factory \
  -H "Content-Type: application/json" \
  -d '{"mode": "ETO", "enable_milestone_billing": true, "require_design_phase": true}'
```

設定後自動調整項目：

- **Pipeline**：MTO 走接單→備料→生產→出貨；MTS 走預測→補貨→出貨；ETO 走 RFQ→設計→里程碑→出貨
- **表單欄位**：MTO 強調交期與工單；MTS 強調安全庫存與補貨點；ETO 強調里程碑與設計文件
- **現金流規則**：MTO 關注採購付款；MTS 關注庫存水位資金佔用；ETO 關注里程碑收款

LLM 問法範例：

中文查詢	說明
「目前工廠是什麼模式？」	查詢目前設定
「幫我改成 MTS 模式，安全庫存 14 天」	變更型態設定
「ETO 模式下開 SO 要注意什麼？」	查詢模式差異

5. 戰情室 (War Room)

戰情室是一個全螢幕的 SVG 價值流儀表板，適合工廠現場多螢幕監控。

開啟方式：

```
# 前端啟動後，打開瀏覽器：
http://localhost:5173/war-room.html
```

戰情室功能：

- 🏢 6 階段水平排列：供應商 → 採購 → 庫存 → 派工 → 品質 → 會計
- 🟦 動畫粒子流：綠色代表物料流、黃色代表金流
- 📡 底部事件串流：即時顯示最近 30 筆事件
- 🔔 節點發光：事件觸發時對應階段發光回饋
- ▶ 模擬事件按鈕：展示用自動模擬事件生成
- 🔄 數據 15 秒自動刷新

6. Provider 切換

LLM-ERP 支援 5 家 LLM Provider，可隨時切換：

```
# 編輯 backend/.env
LLM_PROVIDER=deepseek          # 或 ollama / anthropic / openai / openrouter
LLM_MODEL=deepseek-chat        # 或 gemma4:e4b / claude-sonnet-4 / gpt-4o
MAX_TOOL_ROUNDS=5              # 雲端 5, 本地 8-10
```

切換原則：

- **雲端 API** (DeepSeek / Claude / GPT)：MAX_TOOL_ROUNDS=5，需要 API Key
- **本地模型** (Ollama / Gemma4)：MAX_TOOL_ROUNDS=8-10，不需要 API Key
- **雙語支援**：所有 Provider 皆支援中英文。用中文問就回中文，用英文問就回英文
- 切換後後端會自動重載 (`uvicorn --reload`)

Provider 效能對比 (30 題 Benchmark)

指標	DeepSeek (雲端)	Gemma4 8B (本地 CPU)
	準確率	90% (27/30)
平均回應	7.7s	8.7s
每題成本	~\$0.002	免費
資料主權	外部 API	完全本地

7. 執行 Benchmark

```
cd /mnt/d/Project/LLM_ERP/evaluation

# 執行 30 題標準測試
python3 run_eval.py

# 詳細輸出
python3 run_eval.py --verbose

# 指定輸出檔
python3 run_eval.py --output my-results.json
```

8. 常見問題

Q: 後端啟動失敗？

A: 確認 `.env` 中有正確的 API Key。確認 `pip install -r requirements.txt` 已執行。

Q: LLM 回應太慢？

A: DeepSeek 平均 7.7s，屬於正常。如果是本地模型 (Gemma4)，CPU 推理約 8-10s，有 GPU 可降到 1-3s。

Q: 查詢結果不對？

A: 嘗試調整 `MAX_TOOL_ROUNDS` (本地模型需要更多輪數)。也可在 system prompt 中增加明確的禁止規則。

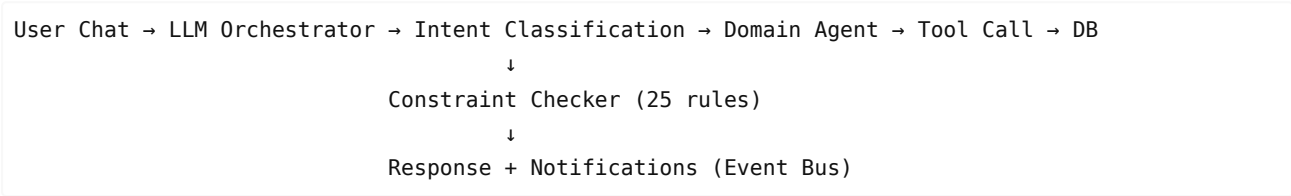
Q: 如何清空資料庫？

A: 使用內建工具：`cd backend && python -m scripts.manage_data reset --force`。或直接刪除 `backend/llm_erp.db` 後重新匯入。

Q: 前端 Vite 抓不到新加的 public 檔案？

A: 重啟 Vite dev server：`Ctrl+C` 後重新 `npm run dev`。

9. 系統架構快速參考



層級	技術	說明
前端	React 18 + TypeScript + Vite + Tailwind	角色儀表板、Chat UI、戰情室
後端	Python FastAPI + SQLAlchemy	7 領域服務、42 API 路由
LLM	DeepSeek / Anthropic / OpenAI / Ollama	37 工具定義、Function Calling
事件	Pub/Sub Event Bus	10 事件類型、角色路由通知
資料庫	SQLite (dev) / PostgreSQL (prod)	22 資料表、Alembic 遷移

10. 典型操作場景 Walkthrough

以下是從接單到出貨的完整流程演練，展示 LLM-ERP 的實際操作方式。

場景一：從訂單到出貨（端到端）

情境：客戶訂了 3 台 **CNC-001** 小型 **CNC** 銑床，**05/15** 前要交貨。

步驟	你做	系統回應
① 查庫存	「CNC-001還有庫存嗎？」	庫存不足，只夠 1 台
② BOM檢查	「CNC-001要做3台，幫我查料夠不夠」	缺料：軸承缺4顆、螺絲缺200顆
③ 開採購單	「向大明螺絲買軸承4顆、向大明螺絲買M6x20螺絲200顆」	採購單 P0-20260507-001 已建立
④ 收貨入庫	「P0-20260507-001貨到了，軸承入庫4顆，螺絲入庫200顆」	庫存已更新：軸承:8顆,螺絲:350顆
⑤ 派工生產	「CNC-001做3台，派工生產」	工單 W0-20260507-001 已釋出並排程
⑥ 完工報工	「W0-20260507-001完工了，3台都做好了」	入庫 3 台 CNC-001 工單完成
⑦ 查成本	「這張工單成本多少？」	材料 \$X, 人工 \$Y, 總成本 \$Z

場景二：異常處理 — 機器故障

情境：**CNC-01** 機台故障，正在生產的工單要重新排程。

① 右移重排程 (最短時間恢復)

你 → 「CNC-01故障了，把後面的工序往後推2小時」

系統 → 「已將 CNC-01 上的工序全部右移 2 小時，預計完工時間：05/07 16:00」

② 替代路徑重排程 (如果有備用機台)

你 → 「CNC-01故障，改到CNC-02生產」

系統 → 「已切換至 CNC-02，工單重新排程完成」

③ 急單插隊 (客戶趕貨)

你 → 「客戶急單W0-20260507-001，插隊優先處理」

系統 → 「W0-20260507-001 已排入最前面，預計提前 3 天完成」

場景三：跨模組查詢 — 決策支援

情境：要決定要不要接一張大訂單，需要綜合判斷。

綜合查詢

你 → 「客戶要5台CNC-001、05/20交期，接不接？」

系統 → 「BOM檢查：料夠 → 

產能檢查：CNC-01 05/15前有 40hr 可用 → 

採購交期：軸承交期 3 天 → 05/10 到貨 → 

結論：可以接！預估完工日 05/16」

場景四：品質異常追溯

情境：客戶反應 CNC-001 有品質問題，需要追溯。

① 查不良記錄

你 → 「CNC-001最近有不良品記錄嗎？」

系統 → 「找到 1 筆 NC：批號 LOT-B001，軸承尺寸超差」

② 建立 CAPA

你 → 「幫我對 NC-20260505-001 建立改善對策」

系統 → 「CAPA 已建立：更換軸承供應商，增加入庫全檢」

③ 從採購源頭管控

你 → 「查一下這批軸承是哪個供應商？」

系統 → 「大明螺絲，評分已從 4.2 降至 3.5」

場景五：業務接單到出貨 (CRM 完整流程)

情境：業務經理接到永裕精密的新訂單，從建立客戶→SO→出貨一次完成。

① 查詢/新增客戶

你 → 「查一下永裕精密的資料」

系統 → 「永裕精密工業，A級客戶，聯絡人林經理，信用額度 NT\$500K」

② 開銷售訂單

你 → 「幫永裕精密開 SO：CNC-001 × 5 件，單價 1,500」

系統 → 「SO-20260507-004 已建立 (draft)，總金額 NT\$7,500」

③ 確認 SO → 自動開工單

你 → 「SO-20260507-004 確認下去」

系統 → 「已確認 → 自動開立工單 W0-20260507-003 (生產中)」

④ 出貨 → 自動扣庫存
 你 → 「S0-20260507-004 出貨」
 系統 → 「已出貨，庫存 CNC-001 由 47 件 → 42 件」

⑤ 完成
 你 → 「S0-20260507-004 完成」
 系統 → 「S0-20260507-004 已送達客戶，訂單完成 

⑥ 記錄互動（業務留底）
 你 → 「記錄一下：永裕精密林經理對交期很滿意」
 系統 → 「 客戶互動事件已記錄」

場景六：MTO 機械加工廠 — 急單處理 (Leads → Opportunity → Contract → 急單評估 → SO → Confirm → Ship)

情境：永裕精密來電要插單趕一批 CNC 零件，客戶願意支付 20% 溢價。工廠為 MTO 模式。

步驟	你做	系統回應
① 查 Lead	「查一下永裕精密是既有客戶還是新的 Lead？」	永裕精密工業，既有客戶 A級客戶，聯絡人林經理
② 建立商機	「幫永裕精密建立商機：CNC-001 × 10，金額 15 萬」	商機 OP-20260507-001 已建立 Pipeline 階段：初步接觸
③ 查合約	「永裕精密有有效合約嗎？」	年度合約 CT-2025-001， 2026/12/31 到期， CNC-001 單價 NT\$1,350
④ 急單評估	「幫我評估急單：客戶願付 20% 溢價，加班成本 NT\$2,000，其他訂單延遲罰款 NT\$1,000」	溢價收入：NT\$3,000 加班成本：NT\$2,000 延遲罰款：NT\$1,000 淨效益： NT\$0 → 剛好打平
⑤ 記錄決策	「記錄決策：接受急單，犧牲其他訂單換取客戶關係」	決策已記錄：2026-05-09 接受永裕精密急單 原因：維持 A 級客戶關係
⑥ 開 SO	「幫永裕精密開 SO：CNC-001 × 10，帶入合約價格」	S0-20260509-001 已建立 (draft) 金額 NT\$13,500
⑦ 確認 SO	「S0-20260509-001 確認下去」 → 自動開工單	S0 已確認 → 自動開工單 W0-20260509-001 (急單優先)
⑧ 出貨	「S0-20260509-001 出貨」	已出貨，庫存 CNC-001 由 5 → -5 (負庫存表示 MTO 在製品)
⑨ AAR	「幫我對這個急單做回顧」	預期：提升客戶滿意度 實際：客戶非常滿意 偏差：+ (正) 行動：建立 VIP 急單 SOP

場景七：MTS 電子零件廠 — 合約管理與補貨 (Contract → Renewal → SO from Catalog → Ship)

情境：宏達電子年度合約即將到期，需續約並安排補貨。工廠為 MTS 模式，安全庫存 14 天。

步驟	你做	系統回應
① 查有效合約	「有哪些快要到期的合約？」	3 筆合約 30 天內到期： 宏達電子 (15天後) 永裕精密 (22天後) 大明電子 (28天後)
② 查合約內容	「宏達電子的合約內容？」	年度合約 CT-2025-002

		品項：電容 A-100 × 5,000 pcs 單價：NT\$0.8/pcs 到期日：2026-05-24
③ 續約	「幫我續約宏達電子， 新年度價格降 5%」	續約提案已建立： 電容 A-100 × 5,000 pcs 新單價：NT\$0.76/pcs 新合約 CT-2026-002 (草案)
④ 確認續約	「續約確認」	合約 CT-2026-002 已生效 到期日：2027-05-24
⑤ 查庫存	「電容 A-100 庫存多少？」	庫存：1,200 pcs 安全庫存：2,000 pcs → 不足！
⑥ 開 SO	「幫宏達電子開 SO： 電容 A-100 × 2,000 pcs」	合約價格 NT\$0.76 已自動帶入 SO-20260509-002 (draft) 總金額 NT\$1,520
⑦ 確認 SO	「SO-20260509-002 確認」	SO 已確認 → MTS 庫存出貨
⑧ 出貨	「出貨」	已出貨 庫存：1,200 → -800 (觸發自動補貨)
⑨ 自動補貨	(系統自動)	自動開立採購單 PO-20260509-001 向華通電子買電容 A-100 × 2,800 pcs 預計補回安全庫存以上

場景八：ETO 自動化設備 — 專案里程碑管理 (Lead → Opportunity → Contract with Milestones → SO → Milestone Billing → AAR)

情境：某半導體廠 RFQ 自動化檢測設備，需從設計到安裝全程管理。工廠為 ETO 模式。

步驟	你做	系統回應
① 建立 Lead	「新增 Lead：華亞半導體， 來源：SEMI 展會，聯絡人陳經理」	Lead LE-20260509-001 已建立，評分：70 (高潛力)
② 建立商機	「建立商機：華亞半導體 自動化光學檢測設備 × 1 套」	商機 OP-20260509-002 已建立 階段：需求了解 預估金額：NT\$250 萬
③ 更新階段	「商機進到提案階段」	商機 OP-20260509-002 階段：提案中
④ 議價成功	「商機成交，準備簽約」	商機已標記為「贏單」 進入合約建立流程
⑤ 建合約	「建立專案合約： 華亞半導體檢測設備 NT\$250 萬」	合約 CT-2026-003 已建立 ETO 里程碑條款： ① 設計完成 30% (NT\$75 萬) ② 採購完成 20% (NT\$50 萬) ③ 組裝完成 30% (NT\$75 萬) ④ 客戶驗收 20% (NT\$50 萬)
⑥ 開 SO	「開 SO，帶入里程碑價格」	SO-20260509-003 (draft) 總金額 NT\$250 萬
⑦ 確認 SO	「確認 SO」	SO 已確認 里程碑 ① 啟動：設計階段開始
⑧ 設計完成請款	「里程碑① 設計完成，請款」	里程碑① 已達成 應收帳款 NT\$75 萬已開立 SO 進度：30%
⑨ AAR	「專案完成後幫我做 AAR」	專案 AAR 報告： 預期交期：90 天 實際交期：95 天 (+5 天) 偏差原因：採購延遲

矯正措施：關鍵料件預先備貨
系統更新：ETO 採購前置期 +7 天

本手冊對應 LLM-ERP v0.1.0。更新日期：2026-05-07。